JobSheet - Week 3 - Inheritance , Abstract Class , Interface

Nama : **Raza Mulki Firdaus**

NIM : **251524120**

Kelas : **1D**

Repo GitHub : **https://github.com/zaaraza/Teknik-Pemrograman-**

Instruksi Pengerjaan:

1. Kerjakan 3 soal di bawah ini dengan melengkapi setiap kolom jawaban yang disediakan pada jobsheet ini.
2. Jawaban setiap soal mencakup source code, screenshot hasil dari program yang ditampilkan full screen termasuk taskbar (tambahkan beberapa screenshot jika diperlukan), penjelasan permasalahan dan solusi yang dihadapi, nama teman yang membantu memecahkan masalah (opsional).
3. Dikumpulkan pada Assignment Classroom sesuai dengan deadline yang tertera pada assignment tersebut.
4. Format penamaan file jobsheet: W3_P_<Kelas 1X>_<3 Digit_NIM_Terakhir>.docx/pdf. Contoh: W3_P_1B_001.docx/pdf.
5. Submit semua jawaban dalam bentuk file java pada repository GitHub masing-masing.

No. 1 Inheritance

Soal Praktikum

Dalam latihan ini, sebuah *subclass* bernama **Cylinder** diturunkan dari *superclass* **Circle** seperti yang ditunjukkan pada diagram kelas (di mana terdapat tanda panah yang menunjuk ke atas dari *subclass* ke *superclass*-nya).
 	
Pelajari bagaimana *subclass* Cylinder memanggil *constructor* dari *superclass* (melalui super() dan super(radius)) serta mewarisi variabel dan *method* dari *superclass* Circle. Berikut adalah kode sumber untuk **Circle.java, Cylinder.java** dan **TestCylinder.java**:
 	
1. Circle.java
/**
 * The Circle class models a circle with a radius and color.
 */
public class Circle { // Save as "Circle.java"
  // private instance variable, not accessible from outside this class
  private double radius;

```
    private String color;

    // Constructors (overloaded)
    /** Constructs a Circle instance with default value for radius and color */
    public Circle() { // 1st (default) constructor
        radius = 1.0;
        color = "red";
    }

    /** Constructs a Circle instance with the given radius and default color */
    public Circle(double r) { // 2nd constructor
        radius = r;
        color = "red";
    }

    /** Returns the radius */
    public double getRadius() {
        return radius;
    }

    /** Returns the area of this Circle instance */
    public double getArea() {
        return radius * radius * Math.PI;
    }

    /** * Return a self-descriptive string of this instance in the form of
     * Circle[radius=?,color=?]
     */
    public String toString() {
        return "Circle[radius=" + radius + " color=" + color + "]";
    }
}
```

2. Cylinder.java

```
public class Cylinder extends Circle { // Save as "Cylinder.java"
    private double height; // private variable

    // Constructor with default color, radius and height
    public Cylinder() {
        super(); // call superclass no-arg constructor Circle()
        this.height = 1.0;
    }

    // Constructor with default radius, color but given height
    public Cylinder(double height) {
        super(); // call superclass no-arg constructor Circle()
        this.height = height;
    }

    // Constructor with default color, but given radius, height
    public Cylinder(double radius, double height) {
        super(radius); // call superclass constructor Circle(radius)
        this.height = height;
    }

    // A public method for retrieving the height
```

```
    public double getHeight() {
        return height;
    }

    // A public method for computing the volume of cylinder
    // use superclass method getArea() to get the base area
    public double getVolume() {
        return getArea() * height;
    }
}
```

3. TestCylinder.java

```
public class TestCylinder { // save as "TestCylinder.java"
    public static void main(String[] args) {

        // Declare and allocate a new instance of cylinder
        // with default color, radius, and height
        Cylinder c1 = new Cylinder();
        System.out.println("Cylinder:"
            + " radius=" + c1.getRadius()
            + " height=" + c1.getHeight()
            + " base area=" + c1.getArea()
            + " volume=" + c1.getVolume());

        // Declare and allocate a new instance of cylinder
        // specifying height, with default color and radius
        Cylinder c2 = new Cylinder(10.0);
        System.out.println("Cylinder:"
            + " radius=" + c2.getRadius()
            + " height=" + c2.getHeight()
            + " base area=" + c2.getArea()
            + " volume=" + c2.getVolume());

        // Declare and allocate a new instance of cylinder
        // specifying radius and height, with default color
        Cylinder c3 = new Cylinder(2.0, 10.0);
        System.out.println("Cylinder:"
            + " radius=" + c3.getRadius()
            + " height=" + c3.getHeight()
            + " base area=" + c3.getArea()
            + " volume=" + c3.getVolume());
    }
}
```

A. Instruksi - Modifikasi Class Circle:

1. Modifikasi class Circle, tambahkan:
    - variable color : string
    - Constructor Circle(radius : double, color : string)
    - Getter and setter untuk color
2. Implementasikan class Cylinder! Jelaskan keterhubungannya dengan class Circle!
3. Konsep apa yang diterapkan pada constructor Cylinder? Jelaskan cara kerja, kelebihan dan kekurangannya!

4. Tuliskan sebuah program pengujian (sebut saja TestCylinder) untuk menguji kelas Cylinder yang telah dibuat!

B. Instruksi - Overriding method getArea():

Metode Overriding dan "Super": Subclass **Cylinder** mewarisi metode getArea() dari superclass **Circle**.

- Cobalah melakukan *overriding* metode getArea() pada subclass Cylinder untuk menghitung luas permukaan (2π × radius × tinggi + 2 × luas alas) tabung, alih-alih luas alasnya saja!

Artinya, jika getArea() dipanggil oleh instans **Circle**, maka ia akan mengembalikan luas lingkaran. Namun, jika getArea() dipanggil oleh instans **Cylinder**, ia akan mengembalikan luas permukaan tabung.

Jika Anda melakukan *override* pada getArea() di subclass Cylinder, maka metode getVolume() tidak akan lagi berfungsi dengan benar. Hal ini terjadi karena getVolume() akan menggunakan metode getArea() yang sudah ditimpa (*overridden*) yang ditemukan di kelas yang sama. (Runtime Java hanya akan mencari ke superclass jika tidak dapat menemukan metode tersebut di kelas ini).

- Perbaikilah metode getVolume() tersebut!

Petunjuk: Setelah melakukan overriding pada metode getArea() di subclass Cylinder, Anda dapat memilih untuk memanggil metode getArea() dari superclass Circle dengan cara memanggil super.getArea().

C. Instruksi - Overriding method getArea():

Sediakan metode toString() pada kelas Cylinder, yang menimpa (override) metode toString() yang diwarisi dari superclass Circle, sebagai contoh:

```
@Override
public String toString() { // di dalam kelas Cylinder
   return "Cylinder: subclass of " + super.toString() // menggunakan toString() milik
Circle
         + " height=" + height;
}
```

- Cobalah metode toString() tersebut di dalam kelas TestCylinder!

Source Code
Circle.java

```java
package SoalSatu;

public class Circle { // Save as "Circle.java"
    // private instance variable, not accessible from outside this class
    private double radius;
    private String color;

    // Constructors (overloaded)
    /** Constructs a Circle instance with default value for radius and color */
    public Circle() { // 1st (default) constructor
        radius = 1.0;
        color = "red";
    }

    /** Constructs a Circle instance with the given radius and default color */
    public Circle(double r) { // 2nd constructor
        radius = r;
        color = "red";
    }

    // Konstruktor dengan parameter radius dan color
    public Circle(double r, String c) { // 3rd constructor
        radius = r;
        color = c;
    }

    /** Returns the radius */
    public double getRadius() {
        return radius;
    }

    // getter untuk color
    public String getColor() {
        return color;
    }

    // setter untuk color
    public void setColor(String c) {
        color = c;
    }

    /** Returns the area of this Circle instance */
    public double getArea() {
        return radius * radius * Math.PI;
    }

    /**
     * * Return a self-descriptive string of this instance in the form of
     * Circle[radius=?,color=?]
     */
    public String toString() {
        return "Circle[radius=" + radius + " color=" + color + "]";
    }
}
```

Cylinder.java

```java
package SoalSatu;

public class Cylinder extends Circle { // Save as "Cylinder.java"
    private double height; // private variable
    // Constructor with default color, radius and height

    public Cylinder() {
        super(); // call superclass no-arg constructor Circle()
        this.height = 1.0;
    }

    // Constructor with default radius, color but given height
    public Cylinder(double height) {
        super(); // call superclass no-arg constructor Circle()
        this.height = height;
    }

    // Constructor with default color, but given radius, height
    public Cylinder(double radius, double height) {
        super(radius); // call superclass constructor Circle(radius)
        this.height = height;
    }

    public Cylinder(double radius, String color, double height) {
        super(radius, color); // call superclass constructor Circle(radius, color)
        this.height = height;
    }

    @Override
    public double getArea() {
        return 2 * Math.PI * getRadius() * height
                + 2 * super.getArea();
    }

    // A public method for retrieving the height
    public double getHeight() {
        return height;
    }

    // A public method for computing the volume of cylinder
    // use superclass method getArea() to get the base area
    public double getVolume() {
        return super.getArea() * height;
    }

    @Override
    public String toString() { // di dalam kelas Cylinder
        return "Cylinder: subclass of " + super.toString() + " height=" + height;
    }
}
```

TestCylinder.java

```java
package SoalSatu;

public class TestCylinder { // save as "TestCylinder.java"
    public static void main(String[] args) {
        // Declare and allocate a new instance of cylinder
        // with default color, radius, and height
        Cylinder c1 = new Cylinder();
        System.out.println("Cylinder:"
                + " radius=" + c1.getRadius()
                + " height=" + c1.getHeight()
                + " base area=" + c1.getArea()
                + " volume=" + c1.getVolume()
                + " toString=" + c1.toString());
        // Declare and allocate a new instance of cylinder
        // specifying height, with default color and radius
        Cylinder c2 = new Cylinder(10.0);
        System.out.println("Cylinder:"
                + " radius=" + c2.getRadius()
                + " height=" + c2.getHeight()
                + " base area=" + c2.getArea()
                + " volume=" + c2.getVolume()
                + " toString=" + c2.toString());
        // Declare and allocate a new instance of cylinder
        // specifying radius and height, with default color
        Cylinder c3 = new Cylinder(2.0, 10.0);
        System.out.println("Cylinder:"
                + " radius=" + c3.getRadius()
                + " height=" + c3.getHeight()
                + " base area=" + c3.getArea()
                + " volume=" + c3.getVolume()
                + " toString=" + c3.toString());
    }
}
```

Screenshot Hasil

```
Cylinder: radius=1.0 height=1.0 base area=12.566370614359172 volume=3.141592653589793 toString=Cylinder:
 subclass of Circle[radius=1.0 color=red] height=1.0
Cylinder: radius=1.0 height=10.0 base area=69.11503837897544 volume=31.41592653589793 toString=Cylinder:
 subclass of Circle[radius=1.0 color=red] height=10.0
Cylinder: radius=2.0 height=10.0 base area=150.79644737231007 volume=125.66370614359172 toString=Cylinde
r: subclass of Circle[radius=2.0 color=red] height=10.0
```

Penjelasan Permasalahan dan Solusi

A. Modifikasi class

1.Sudah diterapkan dalam source code, constructor dan getter setter untuk color sudah ditambahkan

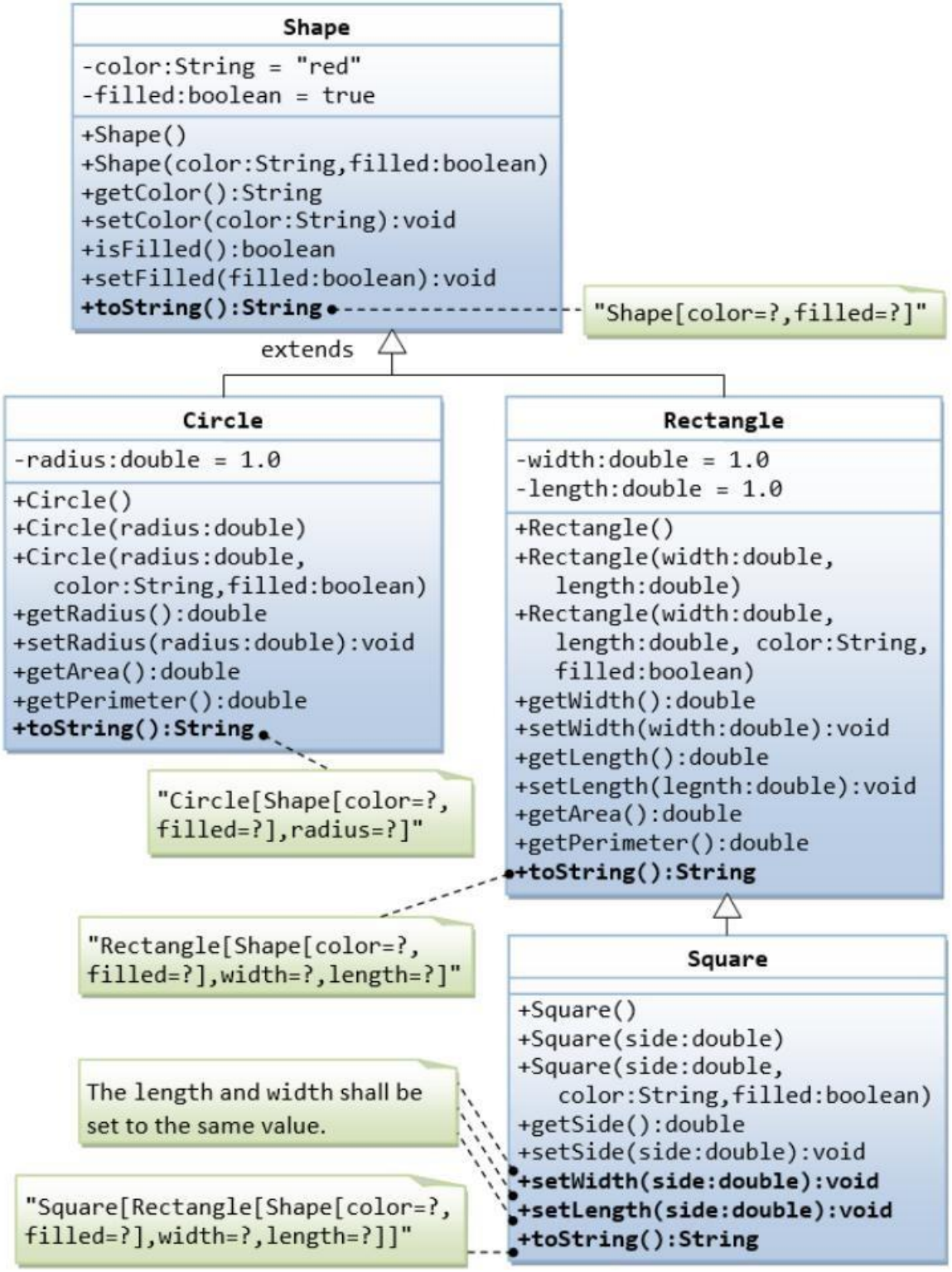
2. Cylinder adalah subclass dari Circle yang menggunakan kata kunci extends. Ini berarti Cylinder mewarisi (inheritance) semua atribut dan method dari Circle (radius, color, getRadius(), getArea(), dll). Cylinder menambahkan atribut height dan method getVolume().

3. Cylinder menggunakan super() dan super(radius) untuk memanggil constructor Circle. Cara kerjanya: ketika objek Cylinder dibuat, constructor Circle dijalankan terlebih dahulu untuk menginisialisasi radius dan color, baru kemudian height diinisialisasi.

	Kelebihan: menghindari duplikasi kode inisialisasi. Dengan menggunakan super() maka akan bisa mengambil semua atribut dari circle seperti radius dan color. 4. Tertera pada source code
B.	Overriding Method Tertera pada source code file **Cylinder.java** Ketika getArea() di-override di Cylinder untuk menghitung luas permukaan, maka getVolume() yang tadinya memanggil getArea() akan ikut menggunakan versi baru (luas permukaan), bukan luas alas. Solusinya: getVolume() memanggil super.getArea() agar tetap menggunakan luas alas (lingkaran) dari superclass Circle.
C.	Method toSring sudah tertera pada source code **Cylinder.java** dan main program untuk mengetes sudah tertera pada source code **TestCylinder.java**
Nama Teman Hal yang Dibantu (Opsional)	

No. 2 Superclass dan Subclass

Soal Praktikum
Superclass Shape dan Subclass-nya: Circle, Rectangle, dan Square.



A. Instruksi - Hirarki Kelas Shape:

1. Buatlah sebuah *superclass* bernama **Shape** (seperti yang ditunjukkan pada diagram kelas), yang berisi:

   - Dua variabel instans: color (String) dan filled (boolean).
   - Dua *constructor*: sebuah *no-arg (no-argument) constructor* yang menginisialisasi color ke "green" dan filled ke true, dan sebuah *constructor* yang menginisialisasi color dan filled ke nilai yang diberikan.
   - *Getter* dan *setter* untuk semua variabel instans. Berdasarkan konvensi, *getter* untuk variabel boolean xxx disebut isXXX() (bukan getXxx() seperti pada tipe lainnya).
   - Sebuah metode toString() yang mengembalikan "A Shape with color of xxx and filled/Not filled".
   - Tulislah sebuah program uji untuk menguji semua metode yang didefinisikan dalam Shape.
2. Buatlah dua *subclass* dari Shape bernama **Circle** dan **Rectangle**, seperti yang ditunjukkan pada diagram kelas.
   Kelas **Circle** berisi:
   - Sebuah variabel instans radius (double).
   - Tiga *constructor* seperti yang ditunjukkan. *No-arg constructor* menginisialisasi radius ke 1.0.
   - *Getter* dan *setter* untuk variabel instans radius.
   - Metode getArea() dan getPerimeter().
   - *Override* metode toString() yang diwarisi, untuk mengembalikan "A Circle with radius=xxx, which is a subclass of yyy", di mana yyy adalah *output* dari metode toString() milik *superclass*.

   Kelas **Rectangle** berisi:
   - Dua variabel instans width (double) dan length (double).
   - Tiga *constructor* seperti yang ditunjukkan. *No-arg constructor* menginisialisasi width dan length ke 1.0.
   - *Getter* dan *setter* untuk semua variabel instans.
   - Metode getArea() dan getPerimeter().
   - *Override* metode toString() yang diwarisi, untuk mengembalikan "A Rectangle with width=xxx and length=zzz, which is a subclass of yyy", di mana yyy adalah *output* dari metode toString() milik *superclass*.
3. Buatlah sebuah kelas bernama **Square**, sebagai *subclass* dari Rectangle. Yakinkan diri Anda bahwa Square dapat dimodelkan sebagai *subclass* dari Rectangle. Square tidak memiliki variabel instans, tetapi mewarisi variabel instans width dan length dari *superclass*-nya, Rectangle.
   - Sediakan *constructor* yang sesuai (seperti yang ditunjukkan dalam diagram kelas)! Petunjuk:

     ```
     public Square(double side) {
       super(side, side);  // Call superclass Rectangle(double, double)
     ```

}

- *Override* metode toString() untuk mengembalikan "A Square with side=xxx, which is a subclass of yyy", di mana yyy adalah *output* dari metode toString() milik *superclass*!
- Apakah Anda perlu melakukan *override* pada getArea() dan getPerimeter()? Cobalah!
- *Override* setLength() dan setWidth() untuk mengubah width sekaligus length, guna menjaga geometri persegi!

Source Code

Shape.Java

```java
// Superclass Shape
public class Shape {
    private String color = "red";
    private boolean filled = true;

    // No-arg constructor
    public Shape() {
        color = "green";
        filled = true;
    }

    // Constructor dengan parameter color dan filled
    public Shape(String color, boolean filled) {
        this.color = color;
        this.filled = filled;
    }

    // Getters and Setters
    public String getColor() {
        return color;
    }

    public void setColor(String color) {
        this.color = color;
    }

    // getter dan setter untuk filled
    public boolean isFilled() {
        return filled;
    }

    public void setFilled(boolean filled) {
        this.filled = filled;
    }

    public String toString() {
        return "A Shape with color of " + color
                + " and " + (filled ? "filled" : "Not filled");
    }
}

```

Circle.Java

```java
//Circle.java
public class Circle extends Shape {
    private double radius = 1.0;

    public Circle() {
        super();
        radius = 1.0;
    }

    public Circle(double radius) {
        super();
        this.radius = radius;
    }

    public Circle(double radius, String color, boolean filled) {
        super(color, filled);
        this.radius = radius;
    }

    public double getRadius() {
        return radius;
    }

    public void setRadius(double radius) {
        this.radius = radius;
    }

    public double getArea() {
        return Math.PI * radius * radius;
    }

    public double getPerimeter() {
        return 2 * Math.PI * radius;
    }

    @Override
    public String toString() {
        return "A Circle with radius=" + radius
                + ", which is a subclass of " + super.toString();
    }
}

```

Rectangle.Java

```java
//Rectangle.java
public class Rectangle extends Shape {
    private double width = 1.0;
    private double length = 1.0;

    public Rectangle() {
        super();
        width = 1.0;
        length = 1.0;
    }

    public Rectangle(double width, double length) {
        super();
        this.width = width;
        this.length = length;
    }

    public Rectangle(double width, double length, String color, boolean filled) {
        super(color, filled);
        this.width = width;
        this.length = length;
    }

    public double getWidth() {
        return width;
    }

    public void setWidth(double width) {
        this.width = width;
    }

    public double getLength() {
        return length;
    }

    public void setLength(double length) {
        this.length = length;
    }

    public double getArea() {
        return width * length;
    }

    public double getPerimeter() {
        return 2 * (width + length);
    }

    @Override
    public String toString() {
        return "A Rectangle with width=" + width + " and length=" + length
                + ", which is a subclass of " + super.toString();
    }
}

```

Square.Java

```java
//Square.java
public class Square extends Rectangle {

    public Square() {
        super(1.0, 1.0);
    }

    public Square(double side) {
        super(side, side);
    }

    public Square(double side, String color, boolean filled) {
        super(side, side, color, filled);
    }

    public double getSide() {
        return getWidth(); // width == length
    }

    public void setSide(double side) {
        setWidth(side);
        setLength(side);
    }

    // Override setWidth dan setLength agar width == length selalu terpenuhi
    @Override
    public void setWidth(double side) {
        super.setWidth(side);
        super.setLength(side);
    }

    @Override
    public void setLength(double side) {
        super.setWidth(side);
        super.setLength(side);
    }

    // getArea() dan getPerimeter() tidak perlu di-override,
    // karena sudah benar dari Rectangle (width == length)

    @Override
    public String toString() {
        return "A Square with side=" + getSide()
                + ", which is a subclass of " + super.toString();
    }
}

```

TestShape.Java

```java
public class TestShape {
    public static void main(String[] args) {

        // ===== Test Shape =====
        System.out.println("===== Shape =====");
        Shape s1 = new Shape();
        System.out.println(s1);
        Shape s2 = new Shape("blue", false);
        System.out.println(s2);

        // ===== Test Circle =====
        System.out.println("\n===== Circle =====");
        Circle c1 = new Circle();
        System.out.println(c1);
        System.out.printf("Area=%.2f, Perimeter=%.2f%n", c1.getArea(), c1.getPerimeter());

        Circle c2 = new Circle(5.0, "yellow", true);
        System.out.println(c2);
        System.out.printf("Area=%.2f, Perimeter=%.2f%n", c2.getArea(), c2.getPerimeter());

        // ===== Test Rectangle =====
        System.out.println("\n===== Rectangle =====");
        Rectangle r1 = new Rectangle();
        System.out.println(r1);
        System.out.printf("Area=%.2f, Perimeter=%.2f%n", r1.getArea(), r1.getPerimeter());

        Rectangle r2 = new Rectangle(4.0, 6.0, "red", false);
        System.out.println(r2);
        System.out.printf("Area=%.2f, Perimeter=%.2f%n", r2.getArea(), r2.getPerimeter());

        // ===== Test Square =====
        System.out.println("\n===== Square =====");
        Square sq1 = new Square();
        System.out.println(sq1);
        System.out.printf("Area=%.2f, Perimeter=%.2f%n", sq1.getArea(), sq1.getPerimeter());

        Square sq2 = new Square(5.0, "purple", true);
        System.out.println(sq2);
        System.out.printf("Area=%.2f, Perimeter=%.2f%n", sq2.getArea(), sq2.getPerimeter());

        // Test setWidth / setLength pada Square
        System.out.println("\n--- Tes setSide pada Square ---");
        sq2.setWidth(10.0);
        System.out.println("After setWidth(10): width=" + sq2.getWidth() + " length=" + sq2.getLength());
        sq2.setLength(7.0);
        System.out.println("After setLength(7): width=" + sq2.getWidth() + " length=" + sq2.getLength());
    }
}
```

Screenshot Hasil

```
===== Shape =====
A Shape with color of green and filled
A Shape with color of blue and Not filled

===== Circle =====
A Circle with radius=1.0, which is a subclass of A Shape with color of green and filled
Area=3.14, Perimeter=6.28
A Circle with radius=5.0, which is a subclass of A Shape with color of yellow and filled
Area=78.54, Perimeter=31.42

===== Rectangle =====
A Rectangle with width=1.0 and length=1.0, which is a subclass of A Shape with color of green and filled
Area=1.00, Perimeter=4.00
A Rectangle with width=4.0 and length=6.0, which is a subclass of A Shape with color of red and Not fill
ed
Area=24.00, Perimeter=20.00

===== Square =====
A Square with side=1.0, which is a subclass of A Rectangle with width=1.0 and length=1.0, which is a sub
class of A Shape with color of green and filled
Area=1.00, Perimeter=4.00
A Square with side=5.0, which is a subclass of A Rectangle with width=5.0 and length=5.0, which is a sub
class of A Shape with color of purple and filled
Area=25.00, Perimeter=20.00

--- Tes setSide pada Square ---
After setWidth(10): width=10.0 length=10.0
After setLength(7): width=7.0 length=7.0
```

Penjelasan Permasalahan dan Solusi

A. Hirarki Kelas Shape 1. Superclass bernama **Shape** sudah dibuat dalam source code 2. Subclass dari Shape bernama **Circle** dan **Rectangle** sudah dibuat dalam source code 3. Subclass **Square** juga sudah dibuat dalam source code Tidak perlu menggunakan override karena setWidth() dan setLength() sudah dioverride sehingga tidak perlu meng-override getArea() dan getPerimeter().
Nama Teman Hal yang Dibantu (Opsional)
-

No. 3 Multiple Inheritance

Soal Praktikum

Dua contoh di bawah merupakan kelas pertama yang mendeskripsikan sekumpulan Karyawan (Employees) yang bekerja di sebuah pabrik, dan kelas kedua yang mendeskripsikan subset Karyawan yang merupakan Manajer (Manager) di pabrik yang sama. Kedua kelas tersebut mewakili sebuah contoh untuk menunjukkan kekuatan ekspresif dari Pewarisan (Inheritance) dalam Java.
 	
Secara khusus, kelas Manager didefinisikan sebagai subclass dari Employee, dengan tujuan untuk menggunakan kembali perangkat lunak yang telah ditulis untuk kelas Employee.
 	
1. Class Employee.java
class Employee {
  private String name;
  private double salary;
  private int hireday;
  private int hiremonth;
  private int hireyear;
 	
  public Employee(String n, double s, int day, int month, int year) {
  name = n;
  salary = s;
  hireday = day;
  hiremonth = month;
  hireyear = year;
  }
 	
  public void print() {
  System.out.println(name + " " + salary + " " + hireYear());
  }
 	
  public void raiseSalary(double byPercent) {
  salary *= 1 + byPercent / 100;
  }
 	
  public int hireYear() {
  return hireyear;
  }
}
 	
2. Class EmployeeTest.java
public class EmployeeTest {
  public static void main(String[] args) {
  // Mendeklarasikan dan mengalokasikan array untuk 3 objek Employee
  Employee[] staff = new Employee[3];
 	
  // Inisialisasi data karyawan
  staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
  staff[1] = new Employee("Maria Bianchi", 2500000, 1, 12, 1991);
  staff[2] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);

```
        // Menaikkan gaji setiap staf sebesar 5%
        for (int i = 0; i < 3; i++) {
            staff[i].raiseSalary(5);
        }

        // Mencetak data dari setiap staf
        for (int i = 0; i < 3; i++) {
            staff[i].print();
        }
    }
}
```

3. Class Manager.java

```
import java.util.Calendar;
import java.util.GregorianCalendar;

class Manager extends Employee {
    private String secretaryName;

    public Manager(String n, double s, int d, int m, int y) {
        super(n, s, d, m, y);
        secretaryName = "";
    }

    @Override
    public void raiseSalary(double byPercent) {
        // Menambahkan bonus 1/2% untuk setiap tahun masa kerja
        GregorianCalendar todaysDate = new GregorianCalendar();
        int currentYear = todaysDate.get(Calendar.YEAR);
        double bonus = 0.5 * (currentYear - hireYear());

        super.raiseSalary(byPercent + bonus);
    }

    public String getSecretaryName() {
        return secretaryName;
    }
}
```

4. Class ManagerTest.java

```
public class ManagerTest {
    public static void main(String[] args) {
        // Mendeklarasikan dan mengalokasikan array untuk 3 objek Employee
        Employee[] staff = new Employee[3];

        // Mengisi array dengan kombinasi Employee dan Manager (Polimorfisme)
        staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
        staff[1] = new Manager("Maria Bianchi", 2500000, 1, 12, 1991);
        staff[2] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);

        // Menaikkan gaji semua staf sebesar 5%
        for (int i = 0; i < 3; i++) {
            staff[i].raiseSalary(5);
        }

        // Mencetak data setiap staf
```

```
        for (int i = 0; i < 3; i++) {
            staff[i].print();
        }
    }
}
```

A. Instruksi 1

Terdapat sebuah abstract class bernama Sortable.

```
abstract class Sortable {
    public abstract int compare(Sortable b);

    public static void shell_sort(Sortable[] a) {
        // Shell sort body
    }
}
```

Ketika Sortable diturunkan ke kelas Employee, metode compare akan diimplementasikan.

```
class Employee extends Sortable {
    /* another methods */

    @Override
    public int compare(Sortable b) {
        Employee eb = (Employee) b;
        if (salary < eb.salary) return -1;
        if (salary > eb.salary) return 1;
        return 0;
    }
}
```

Silakan coba kode di atas! Panggil metode compare, di dalam kelas EmployeeTest!

B. Instruksi 2

Bayangkan kita ingin mengurutkan Manager dengan cara yang serupa:

```
class Managers extends Employee extends Sortable
```

1. Apakah itu akan berhasil?
2. Apa solusi Anda?

Source Code
Employee.Java

```java
//Employee.java extend sortable
public class Employee extends Sortable {
    private String name;
    protected double salary; // protected agar bisa diakses Manager
    private int hireday;
    private int hiremonth;
    private int hireyear;

    public Employee(String n, double s, int day, int month, int year) {
        name = n;
        salary = s;
        hireday = day;
        hiremonth = month;
        hireyear = year;
    }

    public void print() {
        System.out.println(name + " " + salary + " " + hireYear());
    }

    public void raiseSalary(double byPercent) {
        salary *= 1 + byPercent / 100;
    }

    public int hireYear() {
        return hireyear;
    }

    @Override
    public int compare(Sortable b) {
        Employee eb = (Employee) b;
        if (salary < eb.salary)
            return -1;
        if (salary > eb.salary)
            return 1;
        return 0;
    }
}

```

EmployeeTest.Java

```java
public class EmployeeTest {
    public static void main(String[] args) {

        // Array of 3 Employee
        Employee[] staff = new Employee[3];
        staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
        staff[1] = new Employee("Maria Bianchi", 2500000, 1, 12, 1991);
        staff[2] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);

        // Naikkan gaji 5%
        for (int i = 0; i < 3; i++) {
            staff[i].raiseSalary(5);
        }

        System.out.println("=== Data Karyawan Setelah Kenaikan Gaji 5% ===");
        for (int i = 0; i < 3; i++) {
            staff[i].print();
        }

        // Uji shell_sort berdasarkan salary
        System.out.println("\n=== Sebelum Sorting ===");
        for (Employee e : staff)
            e.print();

        Sortable.shell_sort(staff);

        System.out.println("\n=== Sesudah Sorting (ascending by salary) ===");
        for (Employee e : staff)
            e.print();
    }
}
```

Manager.Java

```java
import java.util.Calendar;
import java.util.GregorianCalendar;

public class Manager extends Employee extends Sortable {
    private String secretaryName;

    public Manager(String n, double s, int d, int m, int y) {
        super(n, s, d, m, y);
        secretaryName = "";
    }

    @Override
    public void raiseSalary(double byPercent) {
        GregorianCalendar todaysDate = new GregorianCalendar();
        int currentYear = todaysDate.get(Calendar.YEAR);
        double bonus = 0.5 * (currentYear - hireYear());
        super.raiseSalary(byPercent + bonus);
    }

    public String getSecretaryName() {
        return secretaryName;
    }
}
```

ManagerTest.Java

```java
public class ManagerTest {
    public static void main(String[] args) {

        // Kombinasi Employee dan Manager (Polimorfisme)
        Employee[] staff = new Employee[3];
        staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
        staff[1] = new Manager("Maria Bianchi", 2500000, 1, 12, 1991);
        staff[2] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);

        System.out.println("=== Sebelum Kenaikan Gaji ===");
        for (Employee e : staff) e.print();

        // Naikkan gaji semua staf sebesar 5%
        for (int i = 0; i < 3; i++) {
            staff[i].raiseSalary(5);
        }

        System.out.println("\n=== Setelah Kenaikan Gaji 5% ===");
        System.out.println("(Manager mendapat bonus tambahan berdasarkan tahun kerja)");
        for (int i = 0; i < 3; i++) {
            staff[i].print();
        }

        // Uji shell_sort — Manager IS-A Sortable via Employee
        System.out.println("\n=== Sorting by Salary ===");
        Sortable.shell_sort(staff);
        for (Employee e : staff) e.print();
    }
}
```

Sortable.Java

```java
// Abstrak kelas Sortable yang memiliki metode compare untuk membandingkan dua objek Sortable.
public abstract class Sortable {
    public abstract int compare(Sortable b);

    public static void shell_sort(Sortable[] a) {
        int n = a.length;
        int gap = n / 2;
        while (gap > 0) {
            for (int i = gap; i < n; i++) {
                Sortable temp = a[i];
                int j = i;
                while (j >= gap && a[j - gap].compare(temp) > 0) {
                    a[j] = a[j - gap];
                    j -= gap;
                }
                a[j] = temp;
            }
            gap /= 2;
        }
    }
}
```

Screenshot Hasil

Hasil dari EmployeeTest.java

```
=== Data Karyawan Setelah Kenaikan Gaji 5% ===
Antonio Rossi 2100000.0 1989
Maria Bianchi 2625000.0 1991
Isabel Vidal 3150000.0 1993

=== Sebelum Sorting ===
Antonio Rossi 2100000.0 1989
Maria Bianchi 2625000.0 1991
Isabel Vidal 3150000.0 1993

=== Sesudah Sorting (ascending by salary) ===
Antonio Rossi 2100000.0 1989
Maria Bianchi 2625000.0 1991
Isabel Vidal 3150000.0 1993
```

Hasil dari ManagerTest.Java

```
=== Sebelum Kenaikan Gaji ===
Antonio Rossi 2000000.0 1989
Maria Bianchi 2500000.0 1991
Isabel Vidal 3000000.0 1993

=== Setelah Kenaikan Gaji 5% ===
(Manager mendapat bonus tambahan berdasarkan tahun kerja)
Antonio Rossi 2100000.0 1989
Maria Bianchi 3062500.0 1991
Isabel Vidal 3150000.0 1993

=== Sorting by Salary ===
Antonio Rossi 2100000.0 1989
Maria Bianchi 3062500.0 1991
Isabel Vidal 3150000.0 1993
```

Penjelasan Permasalahan dan Solusi

A. Setelah metode compare ditabahkan, maka ketika dicoba run akan berhasil untuk mencetak println sesudah sorting, sebelumnya tidak bisa, hanya sebelum sorting yang bisa tercetak.
Sebelum :

```
=== Sebelum Sorting ===
Antonio Rossi 2100000.0 1989
Maria Bianchi 2625000.0 1991
Isabel Vidal 3150000.0 1993
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
	The type Employee must implement the inherited abstract method Sortable.compare(Sortable)

	at Employee.compare(Employee.java:4)
	at Sortable.shell_sort(Sortable.java:14)
	at EmployeeTest.main(EmployeeTest.java:38)
```

Sesudah :

```
Antonio Rossi 2100000.0 1989
Maria Bianchi 2625000.0 1991
Isabel Vidal 3150000.0 1993

=== Sebelum Sorting ===
Antonio Rossi 2100000.0 1989
Maria Bianchi 2625000.0 1991
Isabel Vidal 3150000.0 1993

=== Sesudah Sorting (ascending by salary) ===
Antonio Rossi 2100000.0 1989
Maria Bianchi 2625000.0 1991
Isabel Vidal 3150000.0 1993
```

B. 1. Tidak berhasil setelah saya coba run saat menggunakan dua extend (Employee dan Sortable) hasilnya adalah error.

```
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
	Syntax error on token "extends", permits expected

	at Manager.<init>(Manager.java:14)
	at ManagerTest.main(ManagerTest.java:7)
```

2. Setelah saya coba untuk tidak menggunakan extend sortable, saya berhasil untuk me-running programnya, jadi tidak perlu menggunakan 2 extend melainkan satu saja extend Employee (Employee itu sendiri sudah ter-extend dengan Sortable).

Nama Teman Hal yang Dibantu (Opsional)

-